UNIVERSIDAD INTERAMERICANA DE PUERTO RICO · RECINTO DE FAJARDO

PROYECTO FINAL · DESARROLLO WEB FULL-STACK

Sabías que PR

Planificación y Diseño de la Base de Datos

Fase 1: Planificación y Diseño · Fase 2: Diseño de la Base de Datos

Estudiante	William R. Collado
Curso	COMP2053 — Desarrollo Web Full-Stack
Profesora	Milagros Donato Cintrón
Institución	Universidad Interamericana de Puerto Rico, Recinto de Fajardo
Repositorio	github.com/Wcollado72/SabiasquePR-Final
Sitio en línea	sabiasquepr.com
Fecha	24 de octubre de 2026

1. Descripción del proyecto

Sabías que PR es una plataforma web informativa y comunitaria nacida en Puerto Rico con una misión sin fronteras: ayudar a cualquier persona a entender su entorno. Reúne noticias con contexto, contenido educativo, la sección **Revolución Humana** (ensayos, videos y audios con sus propios espacios de debate), una tienda, y un **apoyo voluntario** al proyecto. Todo el contenido es gratis; quien apoya recibe reconocimiento y puede escribir textos más largos en el debate.

El sitio es una aplicación **full-stack**: un frontend en HTML5, CSS3 y JavaScript que consume una **API REST propia** (Node.js y Express), con persistencia en **PostgreSQL** (Supabase), autenticación con **JSON Web Tokens verificados en el servidor** y pagos con **Stripe** (modo de prueba). Está **desplegada y accesible públicamente** en `sabiasquepr.com` (Netlify + Render + Supabase).

2. Objetivo general

Diseñar, desarrollar, probar y desplegar una aplicación web full-stack funcional que integre un frontend responsivo, una API REST propia con operaciones CRUD completas, una base de datos relacional persistente y un sistema de autenticación con JWT verificado en el servidor, aplicando de manera integrada las tecnologías estudiadas en el curso de Desarrollo Web Full-Stack.

Objetivos específicos

- Construir una API REST con rutas organizadas, validación de datos, manejo centralizado de errores y rutas protegidas por rol.
- Diseñar una base de datos relacional de 15 tablas con llaves primarias y foráneas, restricciones y seguridad por fila.
- Implementar registro, inicio de sesión y verificación en dos pasos para el panel de administración.
- Publicar la aplicación en línea con pruebas automáticas y un flujo de integración continua.

3. Justificación

Un proyecto final de desarrollo full-stack evalúa que se construyan y controlen las tres capas de una aplicación real: un backend propio, una base de datos con esquema propio y un frontend que consuma esa API, no solo enlaces a servicios externos. El sitio responde además a una necesidad real: un espacio en español, desde Puerto Rico, donde las noticias vienen con contexto y la comunidad puede debatir con respeto y moderación.

El debate comunitario, la gestión de contenidos y la tienda sirven de eje del CRUD, el criterio de mayor peso de la rúbrica (30 %), y a la vez avanzan la visión del producto: que la comunidad no solo lea, sino que participe.

4. Alcance del sistema

Funcionalidades incluidas en esta entrega:

- **CRUD vía API REST** sobre noticias, contenidos multi-formato, productos y publicaciones del debate; comentarios, reportes de moderación y órdenes con las operaciones que aplican.
- **Autenticación completa:** registro, inicio de sesión, recuperación de contraseña y Google (Supabase Auth); JWT verificado en el servidor; roles usuario, apoyo (premium), moderador y administrador; verificación en dos pasos (TOTP) para el panel de administración.
- **Debate comunitario** en cada sub-sección, con moderación, reportes y filtro automático; límite de palabras según el rol (200 para cuentas gratuitas y 1000 para quienes apoyan).
- **Panel de administración** protegido: formularios CRUD de noticias y contenidos, productos, moderación, métricas de uso de IA y control de gastos de servicios.
- **Agentes de IA** (Investigador, Redactor y Traductor, sobre la API de Claude) para preparar contenido, con registro de costo por uso.
- **Transparencia sobre la IA:** cada noticia o contenido declara si se usó IA y con cuáles herramientas (Claude, Gemini, Perplexity, ChatGPT, Copilot u otras); el público lo ve con iconos.
- **Tienda y apoyo voluntario** con Stripe (checkout y webhook firmado) en modo de prueba.
- **Sitio bilingüe** (español e inglés), diseño responsivo, accesible y con modo claro y oscuro.
- **Calidad:** 1269 pruebas automáticas (Jest y Supertest), ESLint, auditoría de dependencias y de secretos en cada cambio (GitHub Actions) y respaldo diario cifrado de la base de datos.

Fuera de alcance de esta entrega (roadmap de producto, no criterio de evaluación):

- Activar Stripe en modo real (llaves de producción): el flujo está construido y probado con llaves de prueba.
- Lanzamiento público y promoción: se hará cuando haya suficiente contenido en cada sección.
- Debates en vivo tipo chat y nuevas zonas de debate.

5. Tecnologías seleccionadas

Capa	Tecnología	Uso en el proyecto
Frontend	HTML5, CSS3, JavaScript (sin framework)	Sitio responsivo generado por un script de plantillas (build.js), PWA con service worker, modo claro/oscuro, bilingüe
Backend	Node.js 22 + Express 5	API REST propia (54 endpoints) con validadores, middleware de autenticación y manejo de errores centralizado
Base de datos	PostgreSQL (Supabase)	15 tablas relacionales con Row Level Security activo
Autenticación	Supabase Auth + jose (JWKS, ES256)	Registro, inicio de sesión, Google y 2FA en el cliente; verificación del JWT en cada ruta protegida del servidor
Pagos	Stripe	Checkout de tienda y apoyo voluntario, con webhook firmado (modo de prueba)
Servicios	Resend, Cloudflare Turnstile, Beehiiv, Formspree, API de Claude	Correos de autenticación, anti-robots, boletín, formulario de contacto y agentes de IA
Calidad	Jest + Supertest, ESLint	1269 pruebas automáticas y revisión de estilo en cada cambio
Control de versiones	Git + GitHub + GitHub Actions	Historial de commits, ramas y pull requests; flujos de pruebas, auditoría, respaldo y mantenimiento
Despliegue	Netlify + Render + Supabase	Frontend estático (Netlify, dominio propio), API (Render) y base de datos (Supabase)

6. Diagrama de arquitectura

El navegador carga el frontend estático y, por separado, se comunica con Supabase Auth para autenticarse y obtener un JWT firmado. El frontend consulta el backend propio mediante `fetch` enviando ese JWT en el encabezado `Authorization: Bearer`. El backend valida el JWT contra las llaves públicas de Supabase (JWKS) —sin compartir ningún secreto— y solo entonces consulta PostgreSQL.

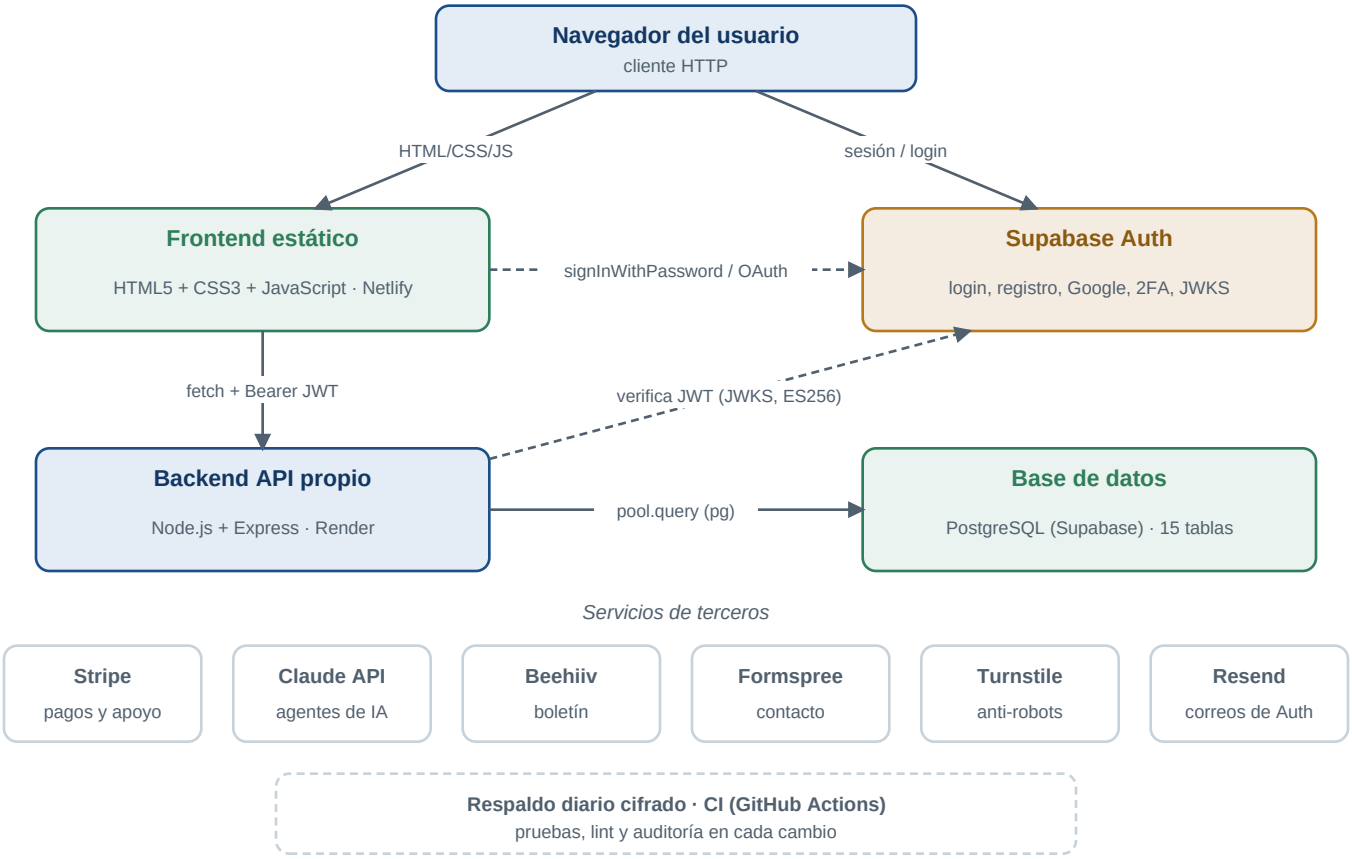


Figura 1. Arquitectura de tres capas con Supabase Auth como servicio de identidad compartido y los servicios de terceros que usa cada capa.

Cuando alguien apoya el proyecto o compra en la tienda, el backend crea una sesión de checkout de Stripe; al completarse el pago, Stripe llama al webhook propio (`POST /api/stripe/webhook`), verificado con la firma de Stripe, que actualiza la orden y el rol del usuario en PostgreSQL sin intervención del frontend.

7. Diagrama general de navegación

Todas las páginas comparten la barra de navegación y el acceso a iniciar sesión. Cada sub-sección de Revolución Humana y Educación tiene su propio debate: se lee sin cuenta y se escribe con sesión. El panel de administración exige el rol de administrador y verificación en dos pasos.

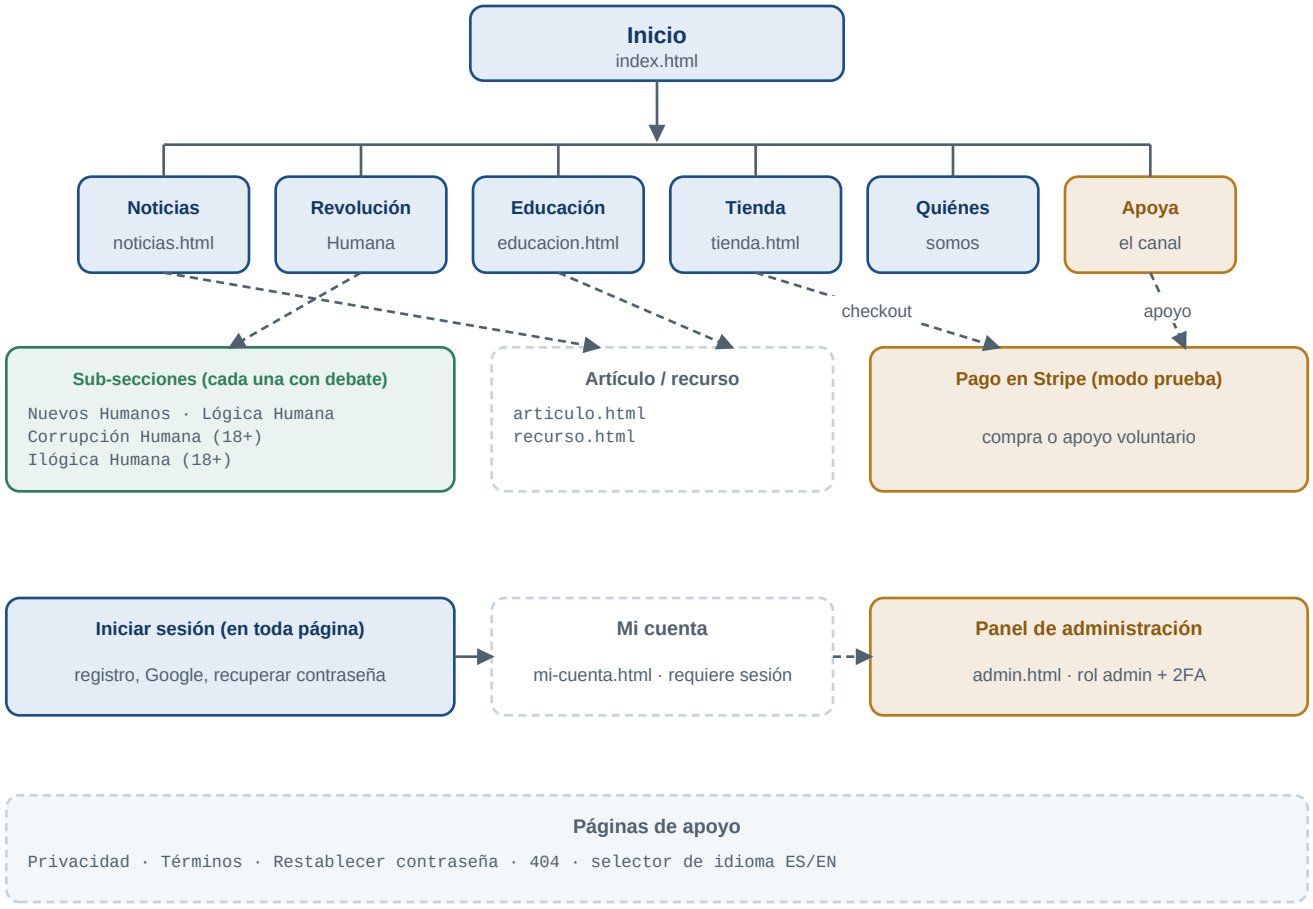


Figura 2. Mapa de navegación: páginas públicas, área con sesión y área restringida por rol.

8. Modelo Entidad-Relación (MER)

La base de datos vive en un proyecto de Supabase (PostgreSQL administrado). `auth.users` es la tabla de identidad que gestiona Supabase Auth; `perfiles` la extiende con el rol y los datos de la cuenta, y es el origen de las relaciones de autoría. El modelo tiene **15 tablas propias**, 15 llaves primarias, **11 llaves foráneas** y Row Level Security activo en 15 de 15.

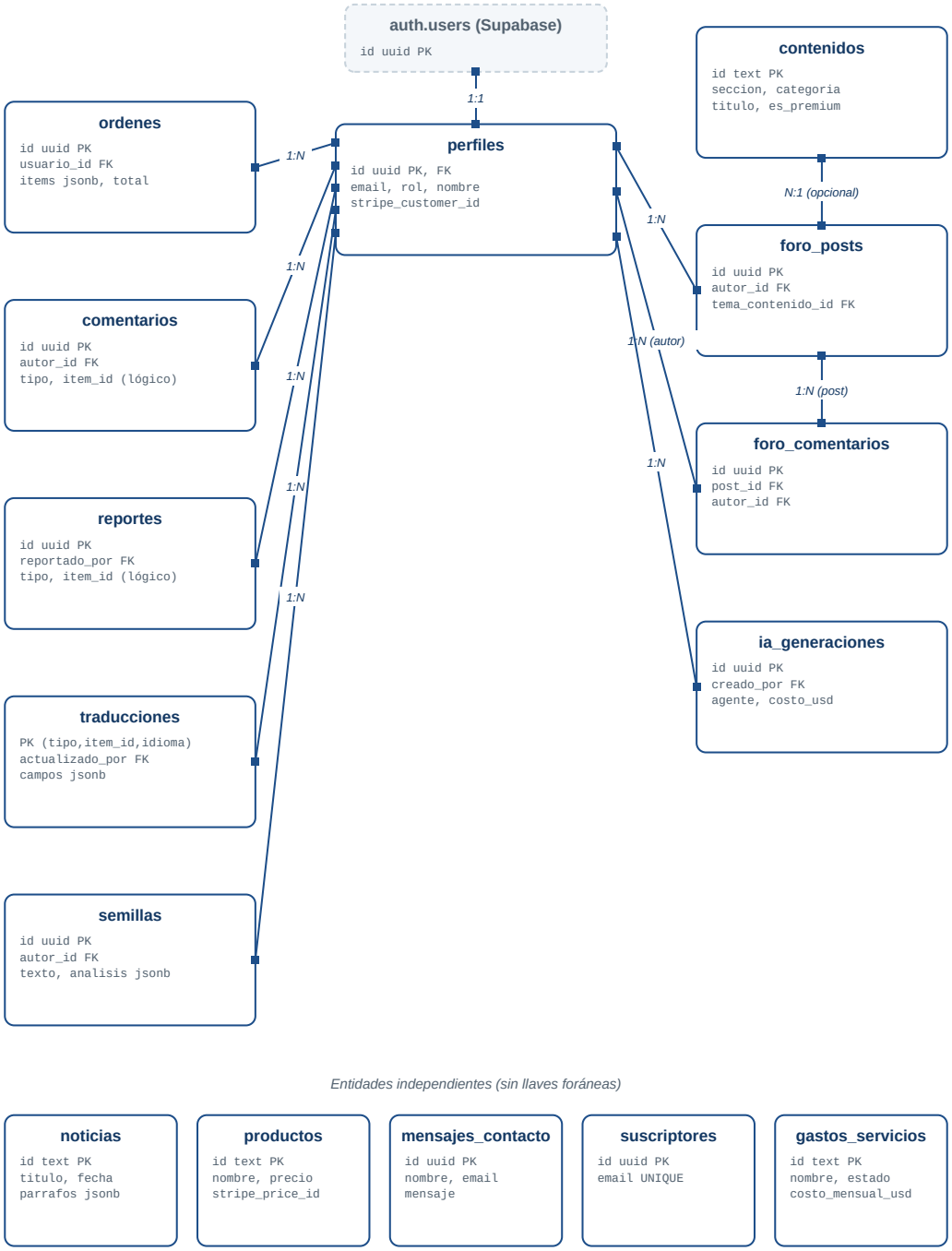


Figura 3. Modelo Entidad-Relación. Las líneas son llaves foráneas; en comentarios, reportes y traducciones la pieza referida se identifica por (tipo, item_id), una relación lógica sin llave foránea porque apunta a tablas distintas según el tipo.

9. Esquema relacional

Notación relacional del modelo: cada tabla con sus atributos, la llave primaria subrayada y las llaves foráneas debajo.

comentarios(id, tipo, item_id, autor_id, cuerpo, creado_en) FK: autor_id → perfiles.id
contenidos(id, seccion, tipo_formato, categoria, titulo, subtitulo_resumen, cuerpo, autor, url_media, fuente_url, fuente_texto, duracion_min, url_portada, portada_alt, fecha_publicacion, destacado, creado_en, subformato, divulgacion_ia, cita_estilo, cita_autor, cita_anio, cita_titulo_original, cita_fecha_acceso, cita_sin_fuente_externa, subseccion, es_premium, enfoque, idioma_original, ia_herramientas, publicado, fuentes)
foro_comentarios(id, post_id, autor_id, cuerpo, creado_en) FK: autor_id → perfiles.id; post_id → foro_posts.id
foro_posts(id, autor_id, titulo, cuerpo, categoria, creado_en, editado_en, subseccion, enfoque, tema_contenido_id, destacado, enlace_media, enlace_media_tipo) FK: autor_id → perfiles.id; tema_contenido_id → contenidos.id
gastos_servicios(id, nombre, estado, costo_mensual_usd, fecha_renovacion, enlace_facturacion, notas, creado_en, actualizado_en)
ia_generaciones(id, agente, fuente_tipo, uso_busqueda_web, busquedas_realizadas, tokens_entrada, tokens_salida, costo_estimado_usd, creado_por, creado_en) FK: creado_por → perfiles.id
mensajes_contacto(id, nombre, email, motivo, mensaje, creado_en)
noticias(id, titulo, fecha, fecha_display, imagen, imagen_alt, imagen_caption, clase, fuente_url, fuente_texto, parrafos, creado_en, categoria, destacado, divulgacion_ia, publicado, idioma_original, ia_herramientas, fuentes)
ordenes(id, usuario_id, items, total, stripe_session_id, estado, creado_en, envio_nombre, envio_direccion, facturacion_direccion) FK: usuario_id → perfiles.id
perfiles(id, email, rol, creado_en, stripe_customer_id, stripe_subscription_id, nombre, edad_confirmada, logica_humana_terminos_ok, fecha_nacimiento, agradecimiento_nombre, agradecimiento_autorizado_en) FK: id → auth.users.id
productos(id, nombre, descripcion, precio, imagen, imagen_alt, stripe_link, disponible, creado_en, stripe_price_id)
reportes(id, tipo, item_id, reportado_por, motivo, estado, creado_en, automatico) FK: reportado_por → perfiles.id
semillas(id, autor_id, titulo, texto, etiquetas, estado, analisis, desarrollos, contenido_id, creado_en, actualizado_en) FK: autor_id → perfiles.id
suscriptores(id, email, creado_en)
traducciones(tipo, item_id, idioma, campos, actualizado_en, actualizado_por) FK: actualizado_por → perfiles.id

10. Diccionario de datos

Cada tabla con sus columnas, tipo de dato, restricciones (llaves, valores por defecto y validaciones) y una descripción. Se genera a partir de la estructura real de la base de datos, por lo que coincide exactamente con el script `backend/sql/schema.sql`.

perfiles

Extiende la identidad de Supabase Auth (auth.users) con el rol, los datos de la cuenta y la información de Stripe. Es el origen de todas las relaciones de autoría.

Campo	Tipo	Restricciones	Descripción
id	uuid	PK, FK → auth.users(id) ON DELETE CASCADE	Mismo identificador que el usuario en Supabase Auth
email	text	NOT NULL	Correo del usuario
rol	text	CHECK IN (usuario, admin, premium, moderador), NOT NULL, DEFAULT 'usuario'	Rol de autorización: usuario, premium (apoyo), moderador o admin
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de creación del perfil
stripe_customer_id	text		Id del cliente en Stripe, cuando apoya
stripe_subscription_id	text		Id de la suscripción activa en Stripe
nombre	text		Nombre visible (opcional)
edad_confirmada	boolean	NOT NULL, DEFAULT false	El usuario confirmó ser mayor de 18 años (acceso a secciones 18+)
logica_humana_terminos_ok	boolean	NOT NULL, DEFAULT false	Aceptó los términos de la sub-sección Lógica Humana
fecha_nacimiento	date		Fecha de nacimiento para verificar la edad (opcional)
agradecimiento_nombre	text	CHECK agradecimiento_nombre IS	Nombre (2 a 40 caracteres) que la persona que apoya autoriza mostrar en el muro de agradecimiento
agradecimiento_autorizado_en	timestamptz		Fecha en que autorizó por primera vez aparecer en el muro de agradecimiento

noticias

Noticias rápidas del sitio (feed de Noticias y RSS). Cada fila es un artículo corto con su fuente.

Campo	Tipo	Restricciones	Descripción
id	text	PK	Identificador (slug generado del título)
titulo	text	NOT NULL	Titular de la noticia
fecha	date	NOT NULL	Fecha editorial

Campo	Tipo	Restricciones	Descripción
fecha_display	text	NOT NULL	Fecha en formato legible
imagen	text	NOT NULL	Ruta o URL de la imagen principal
imagen_alt	text	NOT NULL	Texto alternativo de la imagen (accesibilidad)
imagen_caption	text	NOT NULL	Pie de foto o crédito
clase	text	NOT NULL	Clase CSS para el estilo de la tarjeta
fuente_url	text	NOT NULL	URL de la fuente original
fuente_texto	text	NOT NULL	Nombre visible de la fuente
parrafos	jsonb	NOT NULL, DEFAULT '[]'	Arreglo JSON con los párrafos del cuerpo
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de creación del registro
categoria	text	CHECK IN (11 valores), NOT NULL, DEFAULT 'general'	Categoría temática (filtro del frontend)
destacado	boolean	NOT NULL, DEFAULT false	Pieza destacada (solo una a la vez, índice único parcial)
divulgacion_ia	text	CHECK IN (no, parcial, generado), NOT NULL, DEFAULT 'no'	Transparencia editorial: no, parcial o generado con IA
publicado	boolean	NOT NULL, DEFAULT true	Visible al público; false = borrador
idioma_original	text	CHECK IN (es, en), NOT NULL, DEFAULT 'es'	Idioma en que se redactó: es o en
ia_herramientas	ARRAY	CHECK cardinalityia_herramient, NOT NULL, DEFAULT '{}'[]	Herramientas de IA usadas (claves conocidas o nombres libres), máx. 8
fuentes	jsonb	NOT NULL, DEFAULT '[]'	Otras fuentes de la pieza además de la principal: lista de {nombre, url}, hasta 4

contenidos

Contenido multi-formato (video, ensayo, noticia, lectura corta, audio, foto) que alimenta Noticias, Revolución Humana y Educación.

Campo	Tipo	Restricciones	Descripción
id	text	PK	Identificador (slug generado del título)
seccion	text	CHECK IN (revolucion-humana, educacion, noticias), NOT NULL	Sección donde se publica: noticias, revolucion-humana o educacion
tipo_formato	text	CHECK IN (6 valores), NOT NULL	Formato: video, ensayo, noticia, lectura_corta, audio o foto
categoria	text	CHECK IN (15 valores), NOT NULL, DEFAULT 'general'	Categoría temática
titulo	text	NOT NULL	Título
subtitulo_resumen	text	NOT NULL	Resumen de 2 a 3 oraciones
cuerpo	text		Texto completo (ensayos y lecturas cortas)

Campo	Tipo	Restricciones	Descripción
autor	text		Autor o seudónimo
url_media	text		URL embebible de video o audio
fuente_url	text		URL de la fuente
fuente_texto	text		Nombre visible de la fuente
duracion_min	integer	CHECK duracion_min > 0, NOT NULL, DEFAULT 1	Duración estimada en minutos (mayor que 0)
url_portada	text	NOT NULL	Imagen de portada (ruta o URL)
portada_alt	text	NOT NULL	Texto alternativo de la portada
fecha_publicacion	date	NOT NULL	Fecha de publicación
destacado	boolean	NOT NULL, DEFAULT false	Destacado dentro de su sección (uno a la vez)
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de creación del registro
subformato	text	CHECK IN (11 valores)	Subformato opcional: podcast, entrevista, reportaje, etc.
divulgacion_ia	text	CHECK IN (no, parcial, generado), NOT NULL, DEFAULT 'no'	Transparencia editorial sobre el uso de IA
cita_estilo	text	CHECK IN (10 valores)	Estilo de la cita bibliográfica (APA, MLA, Chicago, etc.)
cita_autor	text		Autor para la cita
cita_anio	text		Año para la cita
cita_titulo_original	text		Título original de la fuente citada
cita_fecha_acceso	date		Fecha en que se consultó la fuente
cita_sin_fuente_externa	boolean	NOT NULL, DEFAULT false	La pieza es redacción propia, sin fuente externa que citar
subseccion	text	CHECK IN (nuevos-humanos, revolucion-mundial, logica-humana, corrupcion-humana, ilogica-humana)	Sub-sección de Revolución Humana (opcional)
es_premium	boolean	NOT NULL, DEFAULT false	Muro de apoyo: solo se muestra el adelanto
enfoque	text	CHECK IN (puerto-rico, estados-unidos, hispanoamerica, internacionales)	Región: puerto-rico, estados-unidos, hispanoamerica o internacionales
idioma_original	text	CHECK IN (es, en), NOT NULL, DEFAULT 'es'	Idioma en que se redactó: es o en
ia_herramientas	ARRAY	CHECK cardinalityia_herramient, NOT NULL, DEFAULT '{}'[]	Herramientas de IA usadas (claves conocidas o nombres libres), máx. 8
publicado	boolean	NOT NULL, DEFAULT true	Borrador (false) o AL AIRE (true): el público solo ve los publicados
fuentes	jsonb	NOT NULL, DEFAULT '[]'	Otras fuentes de la pieza además de la principal: lista de {nombre, url}, hasta 4

productos

Artículos de la tienda. Se leen públicamente; solo el administrador los crea, modifica o elimina.

Campo	Tipo	Restricciones	Descripción
id	text	PK	Identificador (slug generado del nombre)
nombre	text	NOT NULL	Nombre del producto
descripcion	text	NOT NULL	Descripción corta
precio	numeric(10,2)	NOT NULL	Precio en USD
imagen	text	NOT NULL	Ruta o URL de la imagen
imagen_alt	text	NOT NULL	Texto alternativo
stripe_link	text		Enlace de pago de Stripe
disponible	boolean	NOT NULL, DEFAULT true	Visible en la tienda
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de creación del registro
stripe_price_id	text		Id del precio en Stripe para el checkout

ordenes

Compras de la tienda y apoyos procesados por Stripe, con el estado del pago y las direcciones del cliente.

Campo	Tipo	Restricciones	Descripción
id	uuid	PK, DEFAULT gen_random_uuid()	Identificador de la orden
usuario_id	uuid	FK → perfiles(id) ON DELETE SET NULL	Comprador, si inició sesión (se conserva la orden si se borra la cuenta)
items	jsonb	NOT NULL	Lista JSON de productos y cantidades
total	numeric(10,2)	NOT NULL	Total cobrado en USD
stripe_session_id	text		Sesión de pago de Stripe (llave de idempotencia del webhook)
estado	text	CHECK IN (pendiente, pagada, cancelada), NOT NULL, DEFAULT 'pendiente'	Estado del pago: pendiente, pagada o cancelada
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de la orden
envio_nombre	text		Nombre para el envío
envio_direccion	jsonb		Dirección de envío (JSON)
facturacion_direccion	jsonb		Dirección de facturación (JSON)

foro_posts

Publicaciones del debate comunitario, asociadas a una sub-sección y, opcionalmente, a un contenido.

Campo	Tipo	Restricciones	Descripción
id	uuid	PK, DEFAULT gen_random_uuid()	Identificador de la publicación
autor_id	uuid	FK → perfiles(id) ON DELETE CASCADE, NOT NULL	Autor (se borran sus publicaciones si se borra la cuenta)
titulo	text	NOT NULL	Título de la publicación
cuerpo	text	NOT NULL	Contenido (con límite de palabras según el rol)
categoria	text	CHECK IN (general, tecnologia, internacionales, nuevos-humanos, educacion), NOT NULL, DEFAULT 'general'	Categoría temática del foro
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de publicación
editado_en	timestamptz		Fecha de la última edición
subseccion	text	CHECK IN (corrupcion-humana, ilogica-humana, nuevos-humanos, educacion)	Sub-sección donde se debate
enfoque	text	CHECK IN (puerto-rico, estados-unidos, hispanoamerica, internacionales)	Región asociada
tema_contenido_id	text	FK → contenidos(id) ON DELETE SET NULL	Contenido que origina el debate (si se borra, queda en NULL)
destacado	boolean	NOT NULL, DEFAULT false	Publicación destacada por moderación
enlace_media	text		Enlace a imagen, audio o video adjunto
enlace_media_tipo	text	CHECK IN (imagen, audio, video)	Tipo del enlace: imagen, audio o video

foro_comentarios

Comentarios dentro de una publicación del debate.

Campo	Tipo	Restricciones	Descripción
id	uuid	PK, DEFAULT gen_random_uuid()	Identificador del comentario
post_id	uuid	FK → foro_posts(id) ON DELETE CASCADE, NOT NULL	Publicación comentada (se borran con ella)
autor_id	uuid	FK → perfiles(id) ON DELETE CASCADE, NOT NULL	Autor del comentario
cuerpo	text	NOT NULL	Contenido del comentario
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha del comentario

comentarios

Comentarios bajo una noticia o un contenido. La pieza comentada se identifica por (tipo, item_id), sin llave foránea.

Campo	Tipo	Restricciones	Descripción
id	uuid	PK, DEFAULT gen_random_uuid()	Identificador del comentario
tipo	text	CHECK IN (noticia, contenido), NOT NULL	Tipo de pieza comentada: noticia o contenido
item_id	text	NOT NULL	Id de la pieza comentada (relación lógica, sin llave foránea)
autor_id	uuid	FK → perfiles(id) ON DELETE CASCADE, NOT NULL	Autor del comentario
cuerpo	text	NOT NULL	Contenido del comentario
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha del comentario

reportes

Denuncias de publicaciones o comentarios (del debate y de noticias/contenidos), incluidas las que genera el filtro automático, para la moderación.

Campo	Tipo	Restricciones	Descripción
id	uuid	PK, DEFAULT gen_random_uuid()	Identificador del reporte
tipo	text	CHECK IN (debate_post, debate_comentario, articulo_comentario), NOT NULL	Qué se reporta: debate_post, debate_comentario o articulo_comentario
item_id	uuid	NOT NULL	Id de la publicación o comentario reportado
reportado_por	uuid	FK → perfiles(id) ON DELETE CASCADE	Quien reporta (NULL si lo generó el filtro automático)
motivo	text	NOT NULL	Motivo del reporte
estado	text	CHECK IN (pendiente, revisado, descartado), NOT NULL, DEFAULT 'pendiente'	Estado de moderación: pendiente, revisado o descartado
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha del reporte
automatico	boolean	NOT NULL, DEFAULT false	Generado por el filtro automático de palabras

traducciones

Versión en inglés de noticias, contenidos y productos. Llave compuesta (tipo, item_id, idioma).

Campo	Tipo	Restricciones	Descripción
tipo	text	PK, CHECK IN (noticia, contenido, producto)	Tipo de pieza: noticia, contenido o producto
item_id	text	PK	Id de la pieza traducida
idioma	text	PK, CHECK IN (es, en)	Idioma de la traducción (es o en)
campos	jsonb	NOT NULL	Campos traducidos (JSON)
actualizado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de la última actualización

Campo	Tipo	Restricciones	Descripción
actualizado_por	uuid	FK → perfiles(id) ON DELETE SET NULL	Quien la actualizó (NULL si se borra la cuenta)

ia_generaciones

Registro de cada uso de los agentes de IA de la Brigada Coquí (cada uno con su nombre): tokens y costo estimado.

Campo	Tipo	Restricciones	Descripción
id	uuid	PK, DEFAULT gen_random_uuid()	Identificador del registro
agente	text	CHECK IN (12 valores), NOT NULL	Agente que hizo la llamada: investigador, redactor, traductor, clasificadora, resumidora, correctora, guardiana, pintora, grieta, cazadora, contrapunto o farolera
fuente_tipo	text	CHECK IN (url, texto, notas), NOT NULL	Origen del material: url, texto o notas
uso_busqueda_web	boolean	NOT NULL, DEFAULT false	El agente usó búsqueda web
busquedas_realizadas	integer	NOT NULL, DEFAULT 0	Número de búsquedas web hechas
tokens_entrada	integer	NOT NULL, DEFAULT 0	Tokens enviados al modelo
tokens_salida	integer	NOT NULL, DEFAULT 0	Tokens recibidos del modelo
costo_estimado_usd	numeric(10,4)	NOT NULL, DEFAULT 0	Costo estimado de la llamada en USD
creado_por	uuid	FK → perfiles(id) ON DELETE CASCADE, NOT NULL	Administrador que lo ejecutó
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de la llamada

semillas

Cuaderno de ideas propias del autor (Aliado Imperfekto): cada semilla guarda el texto, sus etiquetas, el estado y los análisis de las agentes del Taller.

Campo	Tipo	Restricciones	Descripción
id	uuid	PK, DEFAULT gen_random_uuid()	Identificador de la semilla
autor_id	uuid	FK → perfiles(id) ON DELETE CASCADE, NOT NULL	Autor de la idea (perfil del administrador)
titulo	text	NOT NULL, DEFAULT ''	Título opcional de la idea
texto	text	NOT NULL	La idea tal como la escribió el autor
etiquetas	ARRAY	NOT NULL, DEFAULT '{}'[]	Etiquetas cortas para encontrarla
estado	text	CHECK IN (semilla, explorada, en_desarrollo, lista, pasada), NOT NULL, DEFAULT 'semilla'	Etapa de la idea: semilla, explorada, en_desarrollo, lista o pasada
analisis	jsonb	NOT NULL, DEFAULT '{}'	Último resultado de cada agente del Taller (por nombre), con el hash del texto que leyó

Campo	Tipo	Restricciones	Descripción
desarrollos	jsonb	NOT NULL, DEFAULT '[]'	Borradores que La Farolera escribió a partir de la idea
contenido_id	text		Pieza de Contenidos creada a partir de esta idea, si ya pasó
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de creación
actualizado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de la última edición o análisis

gastos_servicios

Inventario de los servicios de terceros del proyecto (estado del plan, costo mensual y renovación) para el panel de administración.

Campo	Tipo	Restricciones	Descripción
id	text	PK	Identificador (slug del servicio)
nombre	text	NOT NULL	Nombre del servicio (Netlify, Render, Supabase…)
estado	text	CHECK IN (gratis, gratis_por_ahora, por_uso), NOT NULL	Plan: gratis, gratis_por_ahora o por_uso
costo_mensual_usd	numeric(10,2)		Costo mensual en USD, si aplica
fecha_renovacion	date		Próxima renovación
enlace_facturacion	text		Enlace al panel de facturación
notas	text		Notas internas
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de creación del registro
actualizado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de la última actualización

mensajes_contacto

Mensajes recibidos desde el formulario de contacto.

Campo	Tipo	Restricciones	Descripción
id	uuid	PK, DEFAULT gen_random_uuid()	Identificador del mensaje
nombre	text	NOT NULL	Nombre de quien escribe
email	text	NOT NULL	Correo de contacto
motivo	text	NOT NULL	Motivo del mensaje
mensaje	text	NOT NULL	Contenido del mensaje

Campo	Tipo	Restricciones	Descripción
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de envío

suscriptores

Correos suscritos al boletín.

Campo	Tipo	Restricciones	Descripción
id	uuid	PK, DEFAULT gen_random_uuid()	Identificador del suscriptor
email	text	UNIQUE, NOT NULL	Correo suscrito (único)
creado_en	timestamptz	NOT NULL, DEFAULT now()	Fecha de suscripción

11. Normalización, índices y seguridad

El esquema está normalizado a Tercera Forma Normal (3FN): cada tabla describe una sola entidad, sin grupos repetitivos ni dependencias transitivas. Las excepciones son deliberadas: listas ordenadas que no se consultan por separado se guardan como `jsonb` (`noticias.parrafos`, `ordenes.items`, las direcciones de envío y los campos de `traducciones`).

Hay **31 restricciones CHECK** que sustituyen tablas de catálogo (roles, categorías, estados) por ser listas cortas y fijas, y **15 índices** adicionales a las llaves primarias, entre ellos:

- `idx_contenidos_seccion` — filtra y ordena el contenido por sección, categoría y fecha.
- `idx_foro_posts_categoria` y `idx_foro_comentarios_post_id` — cargan el debate por categoría y los comentarios de cada publicación.
- `idx_noticias_un_destacado` e `idx_contenidos_un_destacado` — índices únicos parciales: solo puede haber una pieza destacada.
- `idx_ordenes_stripe_session_id` y `idx_perfiles_stripe_customer_id` — búsquedas rápidas desde el webhook de Stripe.

Las 15 tablas tienen **Row Level Security** activo sin políticas públicas: la API de datos de Supabase queda cerrada a propósito y solo el backend propio, con conexión directa y verificación previa del JWT, accede a los datos.

12. Script de creación de la base de datos

El script completo de creación vive en `backend/sql/schema.sql` y se aplica con `npm run db:schema`. Un script de comprobación (`npm run db:probar-schema`) lo ejecuta en un esquema temporal y compara columnas, restricciones, índices y RLS con la base real y coinciden todos. A continuación se muestra un extracto (el inicio del script, con las tablas principales); el archivo completo está en el repositorio.

```sql
-- Esquema inicial de Sabias que PR
-- Fase 1: solo estructura. Los endpoints CRUD se conectan en la Fase 2.

CREATE TABLE IF NOT EXISTS noticias (
  id              text PRIMARY KEY,
  titulo          text NOT NULL,
  fecha           date NOT NULL,
  fecha_display   text NOT NULL,
  imagen          text NOT NULL,
  imagen_alt      text NOT NULL,
  imagen_caption  text NOT NULL,
  clase           text NOT NULL,
  fuente_url      text NOT NULL,
  fuente_texto    text NOT NULL,
  parrafos        jsonb NOT NULL DEFAULT '[]',
  creado_en       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS productos (
  id              text PRIMARY KEY,
  nombre          text NOT NULL,
  descripcion     text NOT NULL,
  precio          numeric(10,2) NOT NULL,
  imagen          text NOT NULL,
  imagen_alt      text NOT NULL,
  stripe_link     text,
  disponible      boolean NOT NULL DEFAULT true,
  creado_en       timestamptz NOT NULL DEFAULT now()
);

-- Perfil extendido de cada usuario de Supabase Auth (auth.users)
CREATE TABLE IF NOT EXISTS perfiles (
  id              uuid PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  email           text NOT NULL,
  rol             text NOT NULL DEFAULT 'usuario' CHECK (rol IN ('usuario', 'admin')),
  creado_en       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS mensajes_contacto (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  nombre          text NOT NULL,
  email           text NOT NULL,
  motivo          text NOT NULL,
  mensaje         text NOT NULL,
  creado_en       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS suscriptores (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email           text NOT NULL UNIQUE,
  creado_en       timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS ordenes (
  id                  uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  usuario_id          uuid REFERENCES perfiles(id) ON DELETE SET NULL,
  items               jsonb NOT NULL,
  total               numeric(10,2) NOT NULL,
  stripe_session_id   text,
  estado              text NOT NULL DEFAULT 'pendiente' CHECK (estado IN ('pendiente', 'pagada', 'cancelada')),
  creado_en           timestamptz NOT NULL DEFAULT now()
);

-- Publicaciones del foro comunitario
CREATE TABLE IF NOT EXISTS foro_posts (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  autor_id        uuid NOT NULL REFERENCES perfiles(id) ON DELETE CASCADE,
  titulo          text NOT NULL,
```

```
  cuerpo          text NOT NULL,
  categoria       text NOT NULL DEFAULT 'general'
                    CHECK (categoria IN ('general','tecnologia','internacionales','nuevos-humanos','educacion')),
  creado_en       timestamptz NOT NULL DEFAULT now(),
  editado_en      timestamptz
);

-- Comentarios de cada publicacion
CREATE TABLE IF NOT EXISTS foro_comentarios (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  post_id         uuid NOT NULL REFERENCES foro_posts(id) ON DELETE CASCADE,
  autor_id        uuid NOT NULL REFERENCES perfiles(id) ON DELETE CASCADE,
  cuerpo          text NOT NULL,
  creado_en       timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS idx_foro_posts_categoria      ON foro_posts(categoria, creado_en DESC);
CREATE INDEX IF NOT EXISTS idx_foro_comentarios_post_id  ON foro_comentarios(post_id, creado_en ASC);

-- Suscripciones premium (Stripe). Se agregan via ALTER porque
-- CREATE TABLE IF NOT EXISTS no modifica una tabla ya existente.
ALTER TABLE perfiles ADD COLUMN IF NOT EXISTS stripe_customer_id     text;
ALTER TABLE perfiles ADD COLUMN IF NOT EXISTS stripe_subscription_id text;

-- Ampliar el rol permitido para incluir 'premium'.
ALTER TABLE perfiles DROP CONSTRAINT IF EXISTS perfiles_rol_check;
ALTER TABLE perfiles ADD CONSTRAINT perfiles_rol_check
  CHECK (rol IN ('usuario', 'admin', 'premium'));

-- Rol moderador: puede revisar/actuar sobre reportes y borrar contenido
-- reportado del debate (foro_posts/foro_comentarios), pero NO tiene acceso
-- a la gestion de contenido editorial (CRUD de noticias/contenidos/
-- productos) ni al autocompletar con IA -- eso sigue siendo admin-only.
ALTER TABLE perfiles DROP CONSTRAINT IF EXISTS perfiles_rol_check;
ALTER TABLE perfiles ADD CONSTRAINT perfiles_rol_check
  CHECK (rol IN ('usuario', 'admin', 'premium', 'moderador'));

CREATE UNIQUE INDEX IF NOT EXISTS idx_perfiles_stripe_customer_id
  ON perfiles(stripe_customer_id) WHERE stripe_customer_id IS NOT NULL;

-- Categoria de cada noticia (separada de 'clase', que es solo para
-- estilos CSS). Permite filtrar el feed de Noticias por tema.
ALTER TABLE noticias ADD COLUMN IF NOT EXISTS categoria text NOT NULL DEFAULT 'general';

-- (El CHECK de esta columna se define mas abajo, junto al rename
-- 'crimenes' -> 'crimen': definirlo tambien aqui con la lista vieja
-- rompia cada re-corrida del script en cuanto existia una fila con el
-- valor nuevo.)

CREATE INDEX IF NOT EXISTS idx_noticias_categoria ON noticias(categoria, fecha DESC);

-- Noticia destacada de la portada. Solo puede haber una a la vez: el indice
-- unico parcial lo garantiza a nivel de base de datos (el backend limpia la
-- anterior dentro de la misma transaccion antes de marcar la nueva).
ALTER TABLE noticias ADD COLUMN IF NOT EXISTS destacado boolean NOT NULL DEFAULT false;

CREATE UNIQUE INDEX IF NOT EXISTS idx_noticias_un_destacado
  ON noticias (destacado) WHERE destacado;

-- La pagina "Critica Internacional" se retiro; sus temas se fusionaron
-- en la categoria "internacionales". Reasignar publicaciones existentes
-- antes de endurecer el CHECK (el CREATE TABLE de arriba no modifica una
-- tabla ya existente).
UPDATE foro_posts SET categoria = 'internacionales' WHERE categoria = 'critica-internacional';

ALTER TABLE foro_posts DROP CONSTRAINT IF EXISTS foro_posts_categoria_check;
ALTER TABLE foro_posts ADD CONSTRAINT foro_posts_categoria_check
  CHECK (categoria IN ('general', 'tecnologia', 'internacionales', 'nuevos-humanos', 'educacion'));
```

```
-- Contenido multi-formato (video/ensayo/noticia/lectura_corta) para las
-- secciones "Revolución Humana" y "Educación", que antes eran cajas de
-- texto estáticas. Tabla separada de `noticias`: esta última se mantiene
-- 100% periodística (RSS, tono de noticia), mientras que `contenidos`
-- alimenta secciones editoriales de formato mixto.
CREATE TABLE IF NOT EXISTS contenidos (
  id                text PRIMARY KEY,                    -- slug
  seccion           text NOT NULL
                      CHECK (seccion IN ('revolucion-humana', 'educacion')),
  tipo_formato      text NOT NULL
                      CHECK (tipo_formato IN ('video', 'ensayo', 'noticia', 'lectura_corta')),
  categoria         text NOT NULL DEFAULT 'general'
                      CHECK (categoria IN (
                        'general', 'internacionales', 'crimenes', 'salud', 'deportes',
                        'finanzas', 'tecnologia', 'descubrimientos', 'politica', 'clima', 'cultura'
                      )),
  titulo            text NOT NULL,
  subtitulo_resumen text NOT NULL,                       -- dek: 2-3 oraciones, redacción propia
  cuerpo            text,                                -- opcional: texto largo propio (ensayo/lectura_corta)
  autor             text,
  url_media         text,                                -- video embebible (NULL si no aplica)
  fuente_url        text,                                -- enlace a la fuente/medio original
  fuente_texto      text,                                -- nombre visible de esa fuente
  duracion_min      integer NOT NULL DEFAULT 1 CHECK (duracion_min > 0),
  url_portada       text NOT NULL,
  portada_alt       text NOT NULL,
  fecha_publicacion date NOT NULL,
  destacado         boolean NOT NULL DEFAULT false,
  creado_en         timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS idx_contenidos_seccion
  ON contenidos(seccion, categoria, fecha_publicacion DESC);

-- ... (el resto del script está en backend/sql/schema.sql)
```